CSS Introduction (Detailed Guide for PDF)

What is CSS?

CSS (Cascading Style Sheets) is a language used to style and design web pages. It controls how HTML elements appear on screen, paper, or other media.

CSS handles:

- Colors
- Fonts
- Layout
- Spacing
- Animations
- Responsiveness

History of CSS

- CSS was introduced by Håkon Wium Lie in 1996
- Developed to separate content from design
- Latest version: CSS3 (modular approach)

Why Use CSS?

- Improves website design
- Saves development time
- Ensures consistency across pages
- Supports multiple device layouts (responsive design)
- Reduces code duplication

Types of CSS (Detailed)

1. Inline CSS

- Applied directly to an element
- Highest priority
- Not recommended for large projects

2. Internal CSS

- Defined inside tag
- Used for single-page styling

3. External CSS

- Stored in separate file (.css)
- Best practice for large websites

CSS Syntax (Deep Understanding)

```
selector {
  property: value;
}
```

- Selector → targets HTML element
- Property → what you want to change
- Value → how you want it to look

Example:

```
p {
  color: red;
  font-size: 16px;
}
```

CSS Selectors (Advanced)

Basic Selectors

- Element → p, h1
- Class → .className
- ID → #idName

Group Selector

```
h1, p {
  color: blue;
}
```

Descendant Selector

```
div p {
```

```
  color: green;
}
```

Attribute Selector

```
input[type="text"] {
  border: 1px solid black;
}
```

CSS Specificity

Priority order:

1. Inline CSS
2. ID selector
3. Class selector
4. Element selector

CSS Colors (Detailed)

- Named colors
- HEX → #ff5733
- RGB → rgb(255, 87, 51)
- RGBA → includes transparency
- HSL → hue, saturation, lightness

CSS Units

Absolute Units

- px (pixels)

Relative Units

- %
- em
- rem
- vw (viewport width)
- vh (viewport height)

CSS Box Model (Very Important)

Components:

1. Content
2. Padding
3. Border
4. Margin

```
div {
  width: 200px;
  padding: 10px;
  border: 5px solid black;
  margin: 20px;
}
```

CSS Display Property

Types:

- block
- inline
- inline-block
- none

CSS Positioning

Types:

- static
- relative
- absolute
- fixed
- sticky

Example:

```
div {
  position: absolute;
  top: 10px;
  left: 20px;
}
```

CSS Flexbox (Detailed)

Used for 1D layout (row/column)

Properties:

- display: flex
- flex-direction
- justify-content
- align-items

Example:

```
.container {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

CSS Grid (Detailed)

Used for 2D layout

Example:

```
.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
}
```

CSS Backgrounds

```
body {
  background-color: lightblue;
  background-image: url('image.jpg');
  background-size: cover;
}
```

CSS Borders

```
div {
```

```
  border: 2px solid black;
  border-radius: 10px;
}
```

CSS Text Styling

```
p {
  text-align: center;
  text-transform: uppercase;
  letter-spacing: 2px;
}
```

CSS Fonts

```
body {
  font-family: Arial, sans-serif;
  font-size: 16px;
  font-weight: bold;
}
```

CSS Pseudo-classes

```
a:hover {
  color: red;
}
```

Common:

- :hover
- :active
- :focus

CSS Transitions

```
div {
  transition: all 0.3s ease;
}
```

CSS Animations

```
@keyframes example {
  from {background-color: red;}
```

```
  to {background-color: yellow;}
}
```

Responsive Design

Using media queries:

```
@media (max-width: 600px) {
  body {
    background-color: lightgray;
  }
}
```

Complete Example Project

HTML:

```
<!DOCTYPE html>
<html>
<head>
  <title>CSS Project</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>Hello CSS</h1>
    <p>Learning CSS is fun!</p>
  </div>
</body>
</html>
```

CSS:

```
body {
  margin: 0;
  font-family: Arial;
}

.container {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  height: 100vh;
}

h1 {
  color: darkblue;
}
```

```
p {
  color: gray;
}
```

Advantages of CSS

- Better design control
- Reusable styles
- Faster websites
- Responsive layouts

Disadvantages of CSS

- Browser compatibility issues
- Can be complex in large projects

Best Practices

- Use external CSS
- Keep code clean and organized
- Use meaningful class names
- Avoid inline CSS
- Use comments for clarity